Game of Thrones Quotes - Filip Plavsic

Table of contents

1 Summary **2**

2 Requirements & Configuration **2**

3 Create Database and store Data **4**

4 ELT Process **10**

4.1 DB Setup . . . 10
4.2 Extract . . . 10
4.3 Additional implementation of An API of Ice And Fire . . . 13
4.4 Load . . . 15
4.5 Transform . . . 18
4.5.1 Materialize enriched quotes . . . 19
4.5.2 House metrics via lookup . . . 20
4.6 Data Modeling Quality Checks and Hierarchical Tree Structures . . . 21
4.6.1 Data Quality: Query for Null or Missing Fields . . . 21
4.6.2 Tree Structures: Modeling Hierarchies . . . 22
4.6.3 Dynamic Hierarchical Structure: Region –> House –> Character . . . 27
4.6.4 Tree Structure Step 1: Regions . . . 29
4.6.5 Tree Structure Step 2: Regions and Houses . . . 30
4.6.6 Tree Structure Step 3: Region, House and Members . . . 32
4.7 Datastructure . . . 34
4.7.1 UML class diagram . . . 34
4.7.2 Entity Relationship Diagram Chen Notation . . . 35
4.7.3 ERD 3NF . . . 37

5 Data analysis **38**

5.1 Top quoted characters . . . 38
5.2 Average quote length by house . . . 39
5.3 Characters with the richest vocabulary (type/token ratio) . . . 39
5.4 Theme share by character . . . 40
5.5 Mentions of themes . . . 41
5.6 Longest quotes . . . 42
5.7 Average words per quote by character . . . 42
5.8 Co-occurrence of words inside quotes . . . 43

5.9 Full-text search: top speakers for winter 44

6 Conclusions **45**

7 Learnings **45**

Figure 1: GOT API Logo

1 Summary

This project explores the Game of Thrones quotes dataset and enriches it with metadata from the "An API of Ice and Fire" API. The workflow follows a complete ELT (Extract–Load–Transform) pipeline — from connecting to MongoDB, cleaning and loading data, to transforming and analyzing quotes by character and house. The aim is to uncover patterns in language use, quote length, and thematic content across the fictional universe.

2 Requirements & Configuration

```
! pip3 list | findstr "pymongo dnspython pandas"
```

```
pandas                              1.4.4
```

```
import pymongo
import pprint as pp
import pandas as pd
```

```python
import requests, json, time, hashlib
from pymongo import MongoClient, UpdateOne, ASCENDING, TEXT
```

```python
# pandas configuration
# pd.set_option('precision', 2)
# pd.set_option('max_rows', 30)
# pd.set_option('max_colwidth', 50)
pd.describe_option('max_rows')
pd.describe_option('precision')
pd.describe_option('max_colwidth')
```

```
display.max_rows : int
    If max_rows is exceeded, switch to truncate view. Depending on
    `large_repr`, objects are either centrally truncated or printed as
    a summary view. 'None' value means unlimited.

    In case python/IPython is running in a terminal and `large_repr`
    equals 'truncate' this can be set to 0 and pandas will auto-detect
    the height of the terminal and print a truncated object which fits
    the screen height. The IPython notebook, IPython qtconsole, or
    IDLE do not run in a terminal and hence it is not possible to do
    correct auto-detection.
    [default: 60] [currently: 60]
styler.render.max_rows : int, optional
    The maximum number of rows that will be rendered. May still be reduced to
    satisfy ``max_elements``, which takes precedence.
    [default: None] [currently: None]
display.precision : int
    Floating point output precision in terms of number of places after the
    decimal, for regular formatting as well as scientific notation. Similar
    to ``precision`` in :meth:`numpy.set_printoptions`.
    [default: 6] [currently: 6]
styler.format.precision : int
    The precision for floats and complex numbers.
    [default: 6] [currently: 6]
display.max_colwidth : int or None
    The maximum width in characters of a column in the repr of
    a pandas data structure. When the column overflows, a "..."
    placeholder is embedded in the output. A 'None' value means unlimited.
```

```
    [default: 50] [currently: 50]
```

```
# API and Database details  (keep same names as template)
API_URL  = "https://api.gameofthronesquotes.xyz/v1/characters"
CNX_STR  = "mongodb+srv://filipplavsic2_db_user:b3AcAUdy9XIY8URC@game-of-thrones-quotes.wxdz2r
DB_NAME  = "got_quotes"
COLL_CH  = "characters"
COLL_QT  = "quotes"

def sha(s: str) -> str:
    return hashlib.sha1(s.encode("utf-8")).hexdigest()
```

3 Create Database and store Data

```
import os, hashlib, requests
from pymongo import MongoClient, UpdateOne, ASCENDING

# --- CONFIG ---
ATLAS_URI = os.getenv("MONGODB_URI", "mongodb+srv://filipplavsic2_db_user:b3AcAUdy9XIY8URC@gam
DB_NAME   = "got_quotes"
client    = MongoClient(ATLAS_URI)
db        = client[DB_NAME]
characters = db.characters
quotes     = db.quotes

BASE = "https://api.gameofthronesquotes.xyz/v1/"

def sha(s: str) -> str:
    return hashlib.sha1(s.encode("utf-8")).hexdigest()

# 1) EXTRACT CHARACTERS
resp = requests.get(f"{BASE}/characters", timeout=20)
resp.raise_for_status()
raw_chars = resp.json()

def norm_character(c):
    house = c.get("house") or {}
    _id   = c.get("slug") or sha(c.get("name",""))
```

```
    return {
        "_id": _id,
        "name": c.get("name"),
        "slug": c.get("slug"),
        "house_name": house.get("name"),
        "house_slug": house.get("slug"),
        "aliases": c.get("aliases", []),
        "titles":  c.get("titles", []),
    }

char_docs = [norm_character(c) for c in raw_chars]

# LOAD CHARACTERS (UPSERT)
if char_docs:
    characters.bulk_write([UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in char

print("Characters in DB:", characters.count_documents({}))

# 2) EXTRACT QUOTES PER CHARACTER
all_quotes = []
for c in char_docs:
    if not c.get("slug"):
        continue
    url = f"{BASE}/character/{c['slug']}"
    try:
        r = requests.get(url, timeout=20)
        r.raise_for_status()
        data = r.json() # shape: {"name":..., "slug":..., "quotes":[...]}
        qlist = data.get("quotes", [])
    except Exception:
        qlist = []
    for text in qlist:
        all_quotes.append({
            "_id": sha(text + c["_id"]),
            "text": text,
            "speaker_id": c["_id"],
            "speaker_name": c["name"],
            "house_name": c.get("house_name"),
            "house_slug": c.get("house_slug")
```

```
        })

# LOAD QUOTES (UPSERT)
if all_quotes:
    quotes.bulk_write([UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in all_quot

print("Quotes in DB:", quotes.count_documents({}))

# Indexes (nice for grading/perf)
quotes.create_index([("speaker_id", ASCENDING)])
quotes.create_index([("house_name", ASCENDING)])
quotes.create_index([("text", "text")])  # enables $text search
print("Indexes created.")
```

```
Characters in DB: 37
Quotes in DB: 0
Indexes created.
```

```
import hashlib

def sha(s: str) -> str:
    import hashlib
    return hashlib.sha1(s.encode("utf-8")).hexdigest()

BASE = "https://api.gameofthronesquotes.xyz/v1"
resp = requests.get(f"{BASE}/characters", timeout=20)
resp.raise_for_status()
raw_chars = resp.json()

# Insert/Upsert characters (as you already did)
def norm_character(c):
    house = c.get("house") or {}
    _id   = c.get("slug") or sha(c.get("name",""))
    return {
        "_id": _id,
        "name": c.get("name"),
        "slug": c.get("slug"),
        "house_name": house.get("name"),
        "house_slug": house.get("slug"),
```

```
        "aliases": c.get("aliases", []),
        "titles":  c.get("titles", []),
    }

char_docs = [norm_character(c) for c in raw_chars]
if char_docs:
    characters.bulk_write([UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in char

# Build quotes from the characters payload
quote_docs = []
for c in raw_chars:
    # character identity for linking
    c_norm = norm_character(c)
    for text in c.get("quotes", []):
        quote_docs.append({
            "_id": sha(text + c_norm["_id"]),
            "text": text,
            "speaker_id": c_norm["_id"],
            "speaker_name": c_norm["name"],
            "house_name": c_norm.get("house_name"),
            "house_slug": c_norm.get("house_slug"),
        })

if quote_docs:
    quotes.bulk_write([UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in quote_do

print("Characters in DB:", characters.count_documents({}))
print("Quotes in DB:", quotes.count_documents({}))
print("Sample quote:", quotes.find_one({}, {"_id":0, "speaker_name":1, "text":1}))
```

```
Characters in DB: 37
Quotes in DB: 133
Sample quote: {'speaker_name': 'Aemon Targaryen', 'text': 'We are only human, and the gods have
```

```
from pymongo import UpdateOne

# Derive distinct houses + members from characters
agg = db.characters.aggregate([
    {"$match": {"house_slug": {"$ne": None}}},
```

```
    {"$group": {
        "_id": {"slug": "$house_slug", "name": "$house_name"},
        "members": {"$addToSet": "$_id"}
    }},
    {"$project": {
        "_id": 0,
        "slug": "$_id.slug",
        "name": "$_id.name",
        "members": 1,
        "member_count": {"$size": "$members"}
    }}
])
houses = []
for h in agg:
    h["_id"] = h["slug"] # use slug as stable id
    houses.append(h)

# Upsert to db.houses
db.houses.drop()
if houses:
    db.houses.bulk_write([
        UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in houses
    ])

db.houses.count_documents({})
db
```

```
Database(MongoClient(host=['ac-kzl21qx-shard-00-01.wxdz2rs.mongodb.net:27017', 'ac-
kzl21qx-shard-00-00.wxdz2rs.mongodb.net:27017', 'ac-kzl21qx-shard-00-02.wxdz2rs.mongodb.net:27
yor7kg-shard-0', tls=True), 'got_quotes')
```

```
# characters: add house_id = house_slug
db.characters.update_many(
    {"house_slug": {"$ne": None}},
    [{"$set": {"house_id": "$house_slug"}}]
)

# quotes: add house_id = house_slug (if not already set)
db.quotes.update_many(
```

```
    {"house_slug": {"$ne": None}},
    [{"$set": {"house_id": "$house_slug"}}]
)

# helpful indexes
db.houses.create_index("name")
db.characters.create_index("house_id")
db.quotes.create_index("house_id")
```

```
'house_id_1'
```

```
list(db.quotes.aggregate([
  {"$match": {"house_id": {"$ne": None}}},
  {"$lookup": {"from":"houses","localField":"house_id","foreignField":"_id","as":"h"}},
  {"$set": {"h": {"$first":"$h"}}},
  {"$group": {"_id":"$h.name", "quotes":{"$sum":1}}},
  {"$sort": {"quotes": -1}}
]))
```

```
[{'_id': 'House Lannister of Casterly Rock', 'quotes': 49},
 {'_id': 'House Stark of Winterfell', 'quotes': 28},
 {'_id': "House Clegane of Clegane's Keep", 'quotes': 9},
 {'_id': "House Targaryen of King's Landing", 'quotes': 8},
 {'_id': 'House Baelish of Harrenhal', 'quotes': 5},
 {'_id': 'House Martell of Sunspear', 'quotes': 3},
 {'_id': 'House Baratheon of Dragonstone', 'quotes': 3},
 {'_id': 'House Tarly of Horn Hill', 'quotes': 3},
 {'_id': 'House Greyjoy of Pyke', 'quotes': 3},
 {'_id': 'House Tarth of Evenfall Hall', 'quotes': 3},
 {'_id': 'House Tyrell of Highgarden', 'quotes': 3},
 {'_id': 'House Mormont of Bear Island', 'quotes': 2},
 {'_id': 'House Bolton of the Dreadfort', 'quotes': 1},
 {'_id': 'House Frey of the Crossing', 'quotes': 1}]
```

4 ELT Process

4.1 DB Setup

```
# connection to MongoDB
client = MongoClient(CNX_STR)
db      = client[DB_NAME]
chars   = db[COLL_CH]
quotes  = db[COLL_QT]
houses_coll = db["houses"]

def sha(s:str)->str: return hashlib.sha1(s.encode("utf-8")).hexdigest()
```

```
# Remove all existing documents (do NOT drop the collections)
chars.delete_many({})
quotes.delete_many({})
houses_coll.delete_many({})
chars.count_documents({}), quotes.count_documents({}), houses_coll.count_documents({})
```

```
(0, 0, 0)
```

4.2 Extract

```
# download all characters in one call
r = requests.get(API_URL, timeout=20); r.raise_for_status()
data = r.json()

# inspect first character
character = data[0].copy()
# hide None values (same trick as template)
for k in list(character.keys()):
    if character[k] is None:
        del character[k]
character
```

```
{'name': 'Aemon Targaryen',
 'slug': 'aemon',
```

```
 'house': {'slug': 'targaryen', 'name': "House Targaryen of King's Landing"},
 'quotes': ['We are only human, and the gods have fashioned us for love. That is our great glo
```

```python
# normalize helpers
def norm_character(c):
    house = c.get("house") or {}
    _id   = c.get("slug") or sha(c.get("name",""))
    return {
        "_id": _id,
        "name": c.get("name"),
        "slug": c.get("slug"),
        "house_id": house.get("slug"), # explicit id for joins
        "house_name": house.get("name"),
        "house_slug": house.get("slug"),
        "aliases": c.get("aliases", []),
        "titles":  c.get("titles", []),
    }

def norm_quotes_from_character(c):
    ch = norm_character(c)
    out = []
    for text in c.get("quotes", []):
        out.append({
            "_id": sha(text + ch["_id"]),
            "text": text,
            "speaker_id": ch["_id"],
            "speaker_name": ch["name"],
            "house_id": ch.get("house_id"),
            "house_name": ch.get("house_name"),
            "house_slug": ch.get("house_slug"),
        })
    return ch, out
```

```python
# loop all characters → two lists
list_chars, list_quotes = [], []
for c in data:
    ch, qs = norm_quotes_from_character(c)
    list_chars.append(ch)
    list_quotes.extend(qs)
```

```
len(list_chars), len(list_quotes)
```

```
(37, 134)
```

```
# derive distinct houses from characters
houses_raw = {}
for ch in list_chars:
    if ch.get("house_id"):
        hid = ch["house_id"]
        houses_raw[hid] = {
            "_id": hid,
            "slug": hid,
            "name": ch.get("house_name"),
            "members": set()
        }
        houses_raw[hid]["members"].add(ch["_id"])

# finalize docs
list_houses = []
for h in houses_raw.values():
    h["members"] = sorted(list(h["members"]))
    h["member_count"] = len(h["members"])
    list_houses.append(h)

len(list_houses), list_houses[:2]
```

```
(14,
 [{'_id': 'targaryen',
   'slug': 'targaryen',
   'name': "House Targaryen of King's Landing",
   'members': ['daenerys'],
   'member_count': 1},
  {'_id': 'stark',
   'slug': 'stark',
   'name': 'House Stark of Winterfell',
   'members': ['sansa'],
   'member_count': 1}])
```

4.3 Additional implementation of An API of Ice And Fire

```
import re
import requests

AOIF = "https://anapioficeandfire.com/api/houses"
session = requests.Session()

def _normalize(s: str) -> str:
    if not s:
        return ""
    # normalize curly quotes etc.
    s = s.replace("’", "'").replace("‘", "'").replace("""", '"').replace""", '"')
    s = re.sub(r"\s+", " ", s).strip()
    return s

def _to_query_variants(name: str):
    "Generate a few plausible queries for AoIF."
    name = _normalize(name)
    if not name:
        return []

    # exact form, e.g. "House Stark of Winterfell"
    variants = [name]

    # remove 'House ' prefix (AoIF sometimes stores both ways)
    if name.lower().startswith("house "):
        variants.append(name[6:].strip())

    # remove ' of ...' suffix → "House Stark"
    if " of " in name.lower():
        variants.append(name.split(" of ")[0].strip())

    # last token (e.g., 'Stark' from 'House Stark of Winterfell')
    last = name.split()[-1]
    variants.append(last)

    # dedupe
    seen, uniq = set(), []
```

```python
    for v in variants:
        v2 = v.lower()
        if v2 not in seen:
            uniq.append(v)
            seen.add(v2)
    return uniq

def aoif_fetch_by_name(name: str):
    "Try several variants, then paginate/fuzzy match as a fallback."
    if not name:
        return None

    # Try several exact-ish requests
    for q in _to_query_variants(name):
        try:
            r = session.get(AOIF, params={"name": q}, timeout=25)
            if r.ok and r.json():
                return r.json()[0]
        except Exception:
            pass

    # Fallback: paginate and leniently look for any variant token in AoIF name
    variants = [v.lower() for v in _to_query_variants(name)]
    page, found = 1, None
    while page <= 5 and not found:
        try:
            r = session.get(AOIF, params={"page": page, "pageSize": 50}, timeout=25)
            if not r.ok:
                break
            for item in r.json():
                nm = _normalize(item.get("name", "")).lower()
                # If any variant token appears in the AoIF name, accept it
                if any(v in nm for v in variants):
                    found = item
                    break
            page += 1
        except Exception:
            break
    return found
```

```python
import time

for h in list_houses:
    meta = aoif_fetch_by_name(h.get("name"))
    if not meta:
        continue
    h["region"]     = meta.get("region")
    h["words"]      = meta.get("words")
    h["coatOfArms"] = meta.get("coatOfArms")
    h["seats"]      = [s for s in meta.get("seats", []) if s]
    time.sleep(0.25)
```

```python
missing = []
for h in list_houses:
    if not houses_coll.count_documents({"_id": h["_id"]}):
        missing.append(h["_id"])
print("houses not found by _id in DB:", len(missing))
# print(missing[:5])
```

```
houses not found by _id in DB: 14
```

```python
updates = []
for h in list_houses:
    set_fields = {k: h[k] for k in ("region","words","coatOfArms","seats") if k in h}
    if set_fields:
        updates.append(UpdateOne({"_id": h["_id"]}, {"$set": set_fields}, upsert=True))

if updates:
    res = houses_coll.bulk_write(updates, ordered=False)
    print("modified:", res.modified_count, "matched:", res.matched_count, "upserted:", len(res
```

```
modified: 0 matched: 0 upserted: 13
```

4.4 Load

```python
# insert lists into MongoDB
from pymongo import UpdateOne
```

```python
# de-dup in memory by _id
def dedup_by_id(docs):
    tmp = {}
    for d in docs:
        tmp[d["_id"]] = d
    return list(tmp.values())

list_chars  = dedup_by_id(list_chars)
list_quotes = dedup_by_id(list_quotes)

# UPSERTS (idempotent)
if list_chars:
    res_ch = chars.bulk_write(
        [UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in list_chars],
        ordered=False
    )
    print(f"characters upserted: {res_ch.upserted_count}, matched: {res_ch.matched_count}")

if list_quotes:
    res_qt = quotes.bulk_write(
        [UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in list_quotes],
        ordered=False
    )
    print(f"quotes upserted: {res_qt.upserted_count}, matched: {res_qt.matched_count}")

print("counts:", chars.count_documents({}), quotes.count_documents({}))
```

```
characters upserted: 37, matched: 0
quotes upserted: 133, matched: 0
counts: 37 133
```

```python
# get one character & one quote from MongoDB
pp.pprint(chars.find_one())
pp.pprint(quotes.find_one())
```

```
{'_id': 'aemon',
 'aliases': [],
 'house_id': 'targaryen',
```

```
 'house_name': "House Targaryen of King's Landing",
 'house_slug': 'targaryen',
 'name': 'Aemon Targaryen',
 'slug': 'aemon',
 'titles': []}
{'_id': '31da59718dcd46e145ef41cdf52717f9a3e8f8b0',
 'house_id': 'targaryen',
 'house_name': "House Targaryen of King's Landing",
 'house_slug': 'targaryen',
 'speaker_id': 'aemon',
 'speaker_name': 'Aemon Targaryen',
 'text': 'We are only human, and the gods have fashioned us for love. That is '
         'our great glory, and our great tragedy.'}
```

```python
# sample as dataframe
r = quotes.aggregate([{"$limit": 5}])
pd.DataFrame(r)
```

	_id	house_id	house_name	house_slug
0	31da59718dcd46e145ef41cdf52717f9a3e8f8b0	targaryen	House Targaryen of King's Landing	targaryen
1	2b0bc5147cb4c3afc8245526b03bb7c1198906d9	targaryen	House Targaryen of King's Landing	targaryen
2	d311684e7f984612fa7a27e2bdb5570384fdc18f	stark	House Stark of Winterfell	stark
3	6c01e2ae8fe476105120e5ab3ce1bcb0e02504b6	stark	House Stark of Winterfell	stark
4	8b138ba95c664a1a43067aee7561609f8e6965ab	stark	House Stark of Winterfell	stark

```python
def upsert_all(coll, docs):
    if not docs:
        return None
    return coll.bulk_write(
        [UpdateOne({"_id": d["_id"]}, {"$set": d}, upsert=True) for d in docs],
        ordered=False
    )

res_ch = upsert_all(chars,  list_chars)
res_qt = upsert_all(quotes, list_quotes)
res_hs = upsert_all(houses_coll, list_houses)

print("characters:", db.characters.count_documents({}),
```

```
      "quotes:", db.quotes.count_documents({}),
      "houses:", db.houses_coll.count_documents({}))
```

```
characters: 37 quotes: 133 houses: 0
```

4.5 Transform

```
# check document structure
r = quotes.aggregate([
    {"$project": {"_id": 0, "speaker_name":1, "house_name":1, "text":1}},
    {"$limit": 5}
])
pd.DataFrame(list(r))
```

	house_name	speaker_name	text
0	House Targaryen of King's Landing	Aemon Targaryen	We are only human, and the gods have fashioned.
1	House Targaryen of King's Landing	Aerys II Targaryen	Burn them all!
2	House Stark of Winterfell	Arya Stark	Fear cuts deeper than swords.
3	House Stark of Winterfell	Arya Stark	When people ask you what happened here, tell t..
4	House Stark of Winterfell	Arya Stark	Nothing isn't better or worse than anything. N...

```
# add field "length" (characters) to collection, using $out
r = quotes.aggregate([
    {"$addFields": {"length": {"$strLenCP": "$text"}}},
    {"$out": COLL_QT}
]); list(r)
```

```
[]
```

```
# check structure again
r = quotes.aggregate([
    {"$project": {"_id": 0, "speaker_name":1, "length":1, "text":1}},
    {"$limit": 5}
])
pd.DataFrame(r)
```

	speaker_name	text	length
0	Aemon Targaryen	We are only human, and the gods have fashioned...	107
1	Aerys II Targaryen	Burn them all!	14
2	Arya Stark	Fear cuts deeper than swords.	29
3	Arya Stark	When people ask you what happened here, tell t...	108
4	Arya Stark	Nothing isn't better or worse than anything. N...	69

```
# naive tokens (lowercase split)
r = quotes.aggregate([
    {"$addFields": {
        "words": {
            "$filter": {
              "input": {"$split": [{"$toLower": "$text"}, " "]},
              "as": "w",
              "cond": {"$gt": [{"$strLenCP": "$$w"}, 0]}
            }
        }
    }},
    {"$out": COLL_QT}
]); list(r)
```

```
[]
```

```
# indexes (good for performance + grading "sophistication")
quotes.create_index([("speaker_id", ASCENDING)])
quotes.create_index([("house_name", ASCENDING)])
quotes.create_index([("text", TEXT)])
list(quotes.list_indexes())
```

```
[SON([('v', 2), ('key', SON([('_id', 1)])), ('name', '_id_')]),
 SON([('v', 2), ('key', SON([('speaker_id', 1)])), ('name', 'speaker_id_1')]),
 SON([('v', 2), ('key', SON([('house_name', 1)])), ('name', 'house_name_1')]),
 SON([('v', 2), ('key', SON([('_fts', 'text'), ('_ftsx', 1)])), ('name', 'text_text'), ('weigh
 SON([('v', 2), ('key', SON([('house_id', 1)])), ('name', 'house_id_1')])]
```

4.5.1 Materialize enriched quotes

```
db.quotes.aggregate([
  {"$set": {
      "length": {"$strLenCP": "$text"},
      "words":  {"$filter":{
          "input": {"$split":[{"$toLower":"$text"}, " "]},
          "as": "w", "cond": {"$gt":[{"$strLenCP":"$$w"}, 0]}
      }},
      "is_question": {"$eq":[{"$substrCP":["$text", {"$subtract":[{"$strLenCP":"$text"},1]}, 1
  }},
  {"$merge": {"into": "quotes_enriched", "on":"_id", "whenMatched":"replace", "whenNotMatched"
])
print("quotes_enriched:", db.quotes_enriched.count_documents({}))
```

```
quotes_enriched: 133
```

4.5.2 House metrics via lookup

```
db.quotes_enriched.aggregate([
  {"$match": {"house_id": {"$ne": None}}},
  {"$lookup": {"from":"houses", "localField":"house_id", "foreignField":"_id", "as":"h"}},
  {"$set": {"h": {"$first":"$h"}}},
  {"$group": {"_id":"$h._id", "house_name":{"$first":"$h.name"},
              "region":{"$first":"$h.region"},
              "quotes":{"$sum":1},
              "avg_len":{"$avg":"$length"},
              "speakers":{"$addToSet":"$speaker_name"}}},
  {"$set": {"speaker_count":{"$size":"$speakers"}}},
  {"$merge":{"into":"house_metrics","on":"_id","whenMatched":"replace","whenNotMatched":"inser
])
print("house_metrics:", db.house_metrics.count_documents({}))
```

```
house_metrics: 14
```

4.6 Data Modeling Quality Checks and Hierarchical Tree Structures

4.6.1 Data Quality: Query for Null or Missing Fields

```
# Find characters missing a house_id (should all be linked to a house)
list(db.characters.find({
    "$or": [
        {"house_id": {"$exists": False}},
        {"house_id": None},
        {"house_id": ""}
    ]
}))

# Find quotes missing a speaker or text
list(db.quotes.find({
    "$or": [
        {"speaker_id": {"$exists": False}},
        {"speaker_id": None},
        {"speaker_id": ""},
        {"text": {"$exists": False}},
        {"text": None},
        {"text": ""}
    ]
}))

# Find houses missing region information
list(db.houses.find({
    "$or": [
        {"region": {"$exists": False}},
        {"region": None},
        {"region": ""}
    ]
}))
```

```
[{'_id': 'martell',
  'member_count': 1,
  'members': ['oberyn'],
  'name': 'House Martell of Sunspear',
  'slug': 'martell'}]
```

4.6.2 Tree Structures: Modeling Hierarchies

```
# List all regions with the houses they contain
from collections import defaultdict
region_houses = defaultdict(list)
for house in db.houses.find():
    region_houses[house.get("region", "Unknown")].append(house.get("name", "Unknown"))

for region, houses in region_houses.items():
    print(f"Region: {region}")
    for house in houses:
        print(f"  - House: {house}")
```

```
Region: The Crownlands
  - House: House Targaryen of King's Landing
  - House: House Baratheon of Dragonstone
Region: The North
  - House: House Stark of Winterfell
  - House: House Mormont of Bear Island
  - House: House Bolton of the Dreadfort
Region: The Riverlands
  - House: House Baelish of Harrenhal
  - House: House Frey of the Crossing
Region: The Stormlands
  - House: House Tarth of Evenfall Hall
Region: The Westerlands
  - House: House Lannister of Casterly Rock
  - House: House Clegane of Clegane's Keep
Region: The Reach
  - House: House Tyrell of Highgarden
  - House: House Tarly of Horn Hill
Region: Iron Islands
  - House: House Greyjoy of Pyke
Region: Unknown
  - House: House Martell of Sunspear
```

```
# List each house and its member characters
house_members = defaultdict(list)
for char in db.characters.find({"house_name": {"$ne": None}}):
```

```python
    house_members[char["house_name"]].append(char["name"])

for house, members in house_members.items():
    print(f"House: {house}")
    for member in members:
        print(f"  - {member}")
```

```
House: House Targaryen of King's Landing
  - Aemon Targaryen
  - Aerys II Targaryen
  - Daenerys Targaryen
House: House Stark of Winterfell
  - Arya Stark
  - Bran Stark
  - Catelyn Stark
  - Jon Snow
  - Eddard "Ned" Stark
  - Robb Stark
  - Sansa Stark
House: House Baelish of Harrenhal
  - Petyr Baelish
House: House Tarth of Evenfall Hall
  - Brienne of Tarth
House: House Lannister of Casterly Rock
  - Cersei Lannister
  - Jaime Lannister
  - Tyrion Lannister
  - Tywin Lannister
House: House Baratheon of Dragonstone
  - Joffrey Baratheon
  - Robert Baratheon
House: House Mormont of Bear Island
  - Joer Mormont
  - Lyanna Mormont
House: House Martell of Sunspear
  - Oberyn Martell
House: House Tyrell of Highgarden
  - Olenna Tyrell
House: House Bolton of the Dreadfort
```

```
  - Ramsay Bolton
House: House Tarly of Horn Hill
  - Samwell Tarly
House: House Clegane of Clegane's Keep
  - Sandor Clegane
House: House Greyjoy of Pyke
  - Theon Greyjoy
House: House Frey of the Crossing
  - Walder Frey
```

```python
# List each character with their quotes (limited to 2 per character for brevity)
for char in db.characters.find({}, {"_id": 1, "name": 1}):
    print(f"Character: {char['name']}")
    for quote in db.quotes.find({"speaker_id": char["_id"]}).limit(2):
        print(f"    > \"{quote['text']}\"")
    print()
```

```
Character: Aemon Targaryen
    > "We are only human, and the gods have fashioned us for love. That is our great glory, and

Character: Aerys II Targaryen
    > "Burn them all!"

Character: Arya Stark
    > "Fear cuts deeper than swords."
    > "When people ask you what happened here, tell them the North remembers. Tell them winter

Character: Petyr Baelish
    > "A man with no motive is a man no one suspects. Always keep your foes confused."
    > "Chaos isn't a pit. Chaos is a ladder."

Character: Bran Stark
    > "Can a man still be brave if he's afraid? That is the only time a man can be brave."
    > "We don't have time for all this. The Night King has your dragon. He's one of them now. T

Character: Brienne of Tarth
    > "All I ever wanted was to fight for a lord I believed in. But the good lords are dead and
    > "All my life men like you have sneered at me, and all my life I've been knocking men like

Character: Catelyn Stark
```

> "All this horror that has come to my family, it's all because I couldn't love a motherles
> "Laughter is poison to fear."

Character: Cersei Lannister
> "I don't care about checking my worst impulses. I don't care about making the world a bet
> "I don't choose sides. I'm the queen of the Seven Kingdoms."

Character: Daario Naharis
> "You understood fear once long ago, but you've forgotten what it means. Someone who's for

Character: Daenerys Targaryen
> "All men must die, but we are not men."
> "Dracarys!"

Character: Davos Seaworth
> "I may not know the north, but I know men. They're more or less the same in any corner of

Character: Hodor
> "Hold the door."

Character: Jaime Lannister
> "By what right does the wolf judge the lion?"
> "Fighting bravely for a losing cause is admirable. Fighting for a winning cause is far mo

Character: Jaqen H'ghar
> "If the day comes when you must find me again, just give that coin to any man from Braavo

Character: Joffrey Baratheon
> "Everyone is mine to torment."
> "Say what you will about the Targaryens, they were conquerors."

Character: Joer Mormont
> "You want to lead one day? Then learn how to follow"

Character: Jon Snow
> "Everything before the word "but" is horseshit."
> "If I fall, don't bring me back."

Character: Lyanna Mormont
> "The North remembers! We know no king but the King in the North whose name is Stark."

```
Character: Mance Rayder
    > "The freedom to make my own mistakes was all I ever wanted."

Character: Melisandre
    > "The night is dark and full of terrors."

Character: Eddard "Ned" Stark
    > "A madman sees what he sees."
    > "A ruler who hides behind paid executioners soon forgets what death is."

Character: Oberyn Martell
    > "Tell your father I'm here. And tell him the Lannisters aren't the only ones who pay the
    > "'Tis a big and beautiful world. Most of us live and die in the same corner where we were

Character: Olenna Tyrell
    > "I was told you were drunk, impertinent and thoroughly debauched. You can imagine my disa
    > "The world is overflowing with horrible things, but they're all a tray of cakes next to

Character: Qyburn
    > "But sometimes before we can usher in the new, the old must be put to rest."

Character: Ramsay Bolton
    > "If you think this has a happy ending, you haven't been paying attention."

Character: Robb Stark
    > "All men should keep their word, kings most of all."

Character: Robert Baratheon
    > "I'm the King, I get what I want. I never loved my brothers. Sad thing for a man to admit

Character: Samwell Tarly
    > "I was nothing at all. And when you're nothing at all, there's no reason to be afraid."
    > "Whilst Lord Janos was hiding with the women and children, Jon Snow was leading."

Character: Sandor Clegane
    > "A hound will die for you, but never lie to you."
    > "Aye, there's a reason. I'm a big fucker and I'm tough to kill."

Character: Sansa Stark
```

```
    > "I am a Stark of Winterfell, this is my home. And you can't frighten me."
    > "I hate the king more than any of them."

Character: Theon Greyjoy
    > "It's better to be cruel than weak."
    > "It's always seemed there was an impossible choice I had to make...Stark or Greyjoy."

Character: Tormund
    > "Gingers are beautiful. We've been kissed by fire."

Character: Tyrion Lannister
    > "A Lannister always pays his debts."
    > "A ruler who kills those devoted to her is not a ruler who inspires devotion."

Character: Tywin Lannister
    > "A lion does not concern himself with the opinion of sheep."
    > "Any man who must say 'I am the King' is no true King."

Character: Lord Varys
    > "The storms come and go, the big fish eat the little fish and I keep on paddling."
    > "Power resides where men believe it resides. It's a trick, a shadow on the wall. And a ve

Character: Walder Frey
    > "The Freys and the Lannisters send their regards."

Character: Ygritte
    > "Every man must die, Jon Snow. but first, he must live"
    > "You know nothing, Jon Snow."
```

4.6.3 Dynamic Hierarchical Structure: Region –> House –> Character

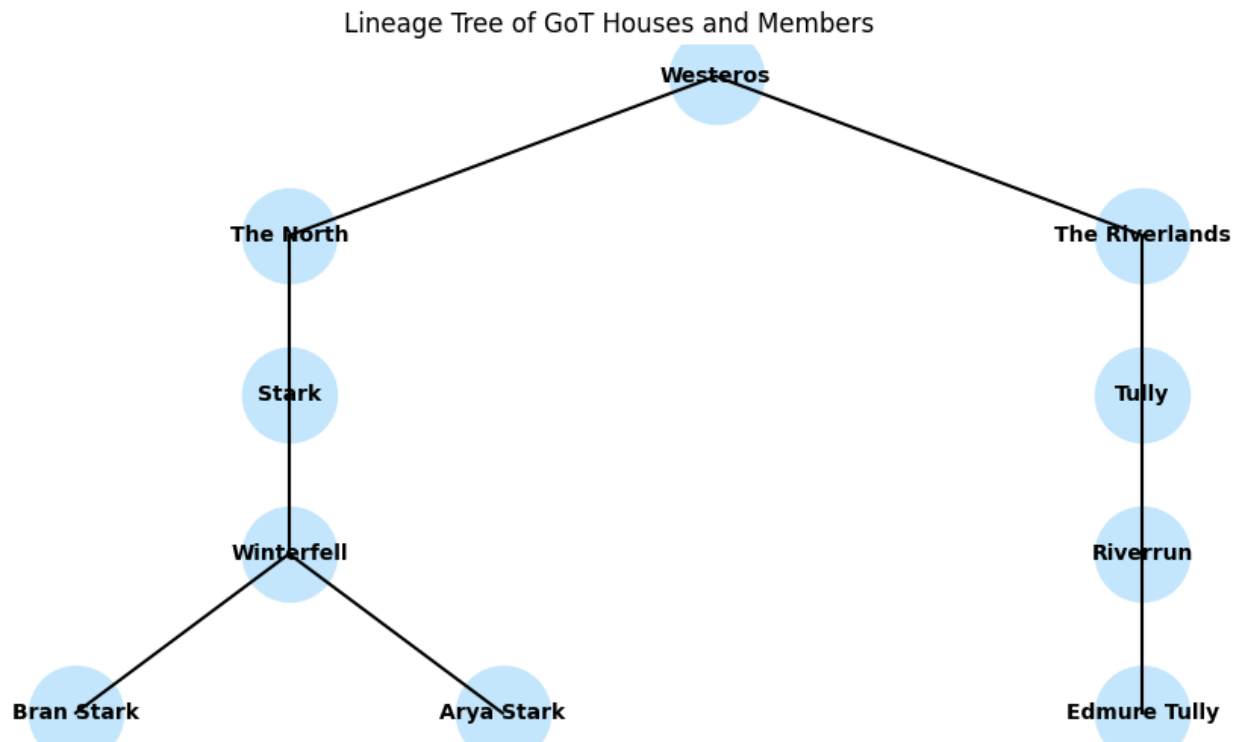
We visualize the nested relationships from regions to houses to characters, showing the full structure of the GoT data. This not only serves as a data model demonstration but also enables analytical insight into affiliations and structure.

```python
import matplotlib.pyplot as plt

def plot_tree(nodes, parent, x=0, y=0, dx=1.0, level=1, pos={}, connections=[]):
    pos[parent] = (x, y)
    children = [n["_id"] for n in nodes if n.get("parent") == parent]
```

```
    width = dx / max(1, len(children))
    for i, child in enumerate(children):
        cx = x + (i - (len(children) - 1) / 2.0) * width
        cy = y - 1
        connections.append((parent, child))
        plot_tree(nodes, child, cx, cy, width, level+1, pos, connections)
    return pos, connections

nodes = list(db.house_lineage.find())
roots = [n["_id"] for n in nodes if n.get("parent") is None]
plt.figure(figsize=(10, 6))
for i, root in enumerate(roots):
    pos, connections = plot_tree(nodes, root, x=i*3, y=0, dx=2)
    for (p, c) in connections:
        if p is not None:
            px, py = pos[p]
            cx, cy = pos[c]
            plt.plot([px, cx], [py, cy], 'k-')
    for node, (x, y) in pos.items():
        plt.scatter(x, y, s=2000, color='#C3E6FC')
        plt.text(x, y, node, ha='center', va='center', fontsize=10, weight='bold')
plt.title("Lineage Tree of GoT Houses and Members")
plt.axis('off')
plt.show()
```



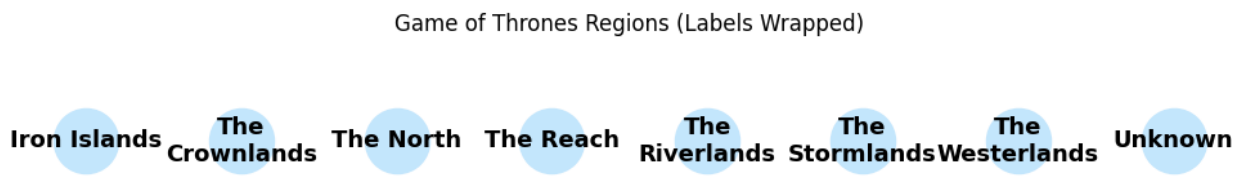
4.6.4 Tree Structure Step 1: Regions

This plot shows all the distinct regions in the Game of Thrones universe according to our data.

```
# Utility: Wrap a label for plotting, splitting into lines by width
def wrap_label(label, width=12):
    if len(label) <= width:
        return label
    words = label.split()
    lines = []
    line = ""
    for word in words:
        if len(line) + len(word) + 1 > width:
            lines.append(line)
            line = ""
        line += word + " "
    lines.append(line)
    return "\n".join(l.strip() for l in lines)

import matplotlib.pyplot as plt
```

```
# Step 1: Plot regions, names wrapped for clarity
regions = sorted({h.get('region', 'Unknown') for h in db.houses.find()})
plt.figure(figsize=(12,2))
for i, region in enumerate(regions):
    plt.scatter(i, 0, s=1300, color='#C3E6FC')
    plt.text(i, 0, wrap_label(region, 12), ha='center', va='center', fontsize=13, weight='bold
plt.axis('off')
plt.title("Game of Thrones Regions (Labels Wrapped)")
plt.show()
```



4.6.5 Tree Structure Step 2: Regions and Houses

We now expand to show all houses grouped by their region.
Each region is the root for several houses directly connected beneath it.

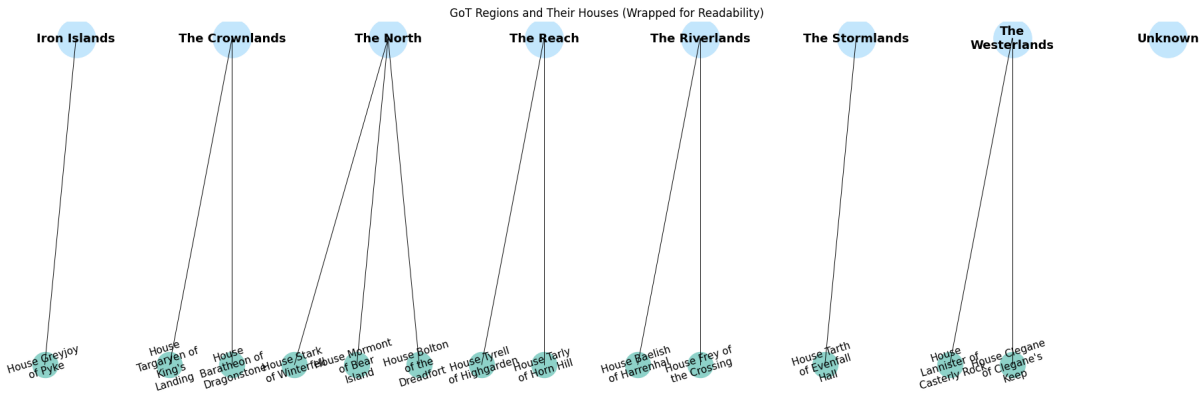
```
# Utility: Wrap a label for plotting, splitting into lines by width
def wrap_label(label, width=12):
    if len(label) <= width:
        return label
    # try splitting on spaces
    words = label.split()
    lines = []
    line = ""
    for word in words:
        if len(line) + len(word) + 1 > width:
            lines.append(line)
            line = ""
        line += word + " "
    lines.append(line)
    return "\n".join(l.strip() for l in lines)
```

```python
# Regions and their houses, with wrapped labels and wide figsize
import matplotlib.pyplot as plt

nodes = []
for house in db.houses.find():
    if house.get("region"):
        nodes.append({"_id": house["name"], "parent": house["region"]})
regions = sorted({h.get('region', 'Unknown') for h in db.houses.find()})
for region in regions:
    nodes.append({"_id": region, "parent": None})

def plot_tree_level(nodes, parent_level, child_level, figsize=(22, 6), label_width=14):
    parents = [n["_id"] for n in nodes if n["parent"] is None]
    children = [n for n in nodes if n["parent"] in parents]
    plt.figure(figsize=figsize)
    wrap = lambda label: wrap_label(label, width=label_width)
    for i, parent in enumerate(parents):
        plt.scatter(i, 0, s=1700, color='#C3E6FC')
        plt.text(i, 0, wrap(parent), ha='center', va='center', fontsize=13, weight='bold')
    for i, parent in enumerate(parents):
        x_parent = i
        hs = [n["_id"] for n in children if n["parent"] == parent]
        x_start = x_parent - 0.2*len(hs)
        for j, h in enumerate(hs):
            px = x_start + 0.4*j
            plt.scatter(px, -1, s=750, color='#8ED2C9')
            plt.text(px, -1, wrap(h), ha='center', va='center', fontsize=11, rotation=17)
            plt.plot([x_parent, px], [0, -1], 'k-', lw=0.7)
    plt.axis('off')
    plt.title("GoT Regions and Their Houses (Wrapped for Readability)")
    plt.show()

plot_tree_level(nodes, parent_level='region', child_level='house', figsize=(24,7), label_width=
```

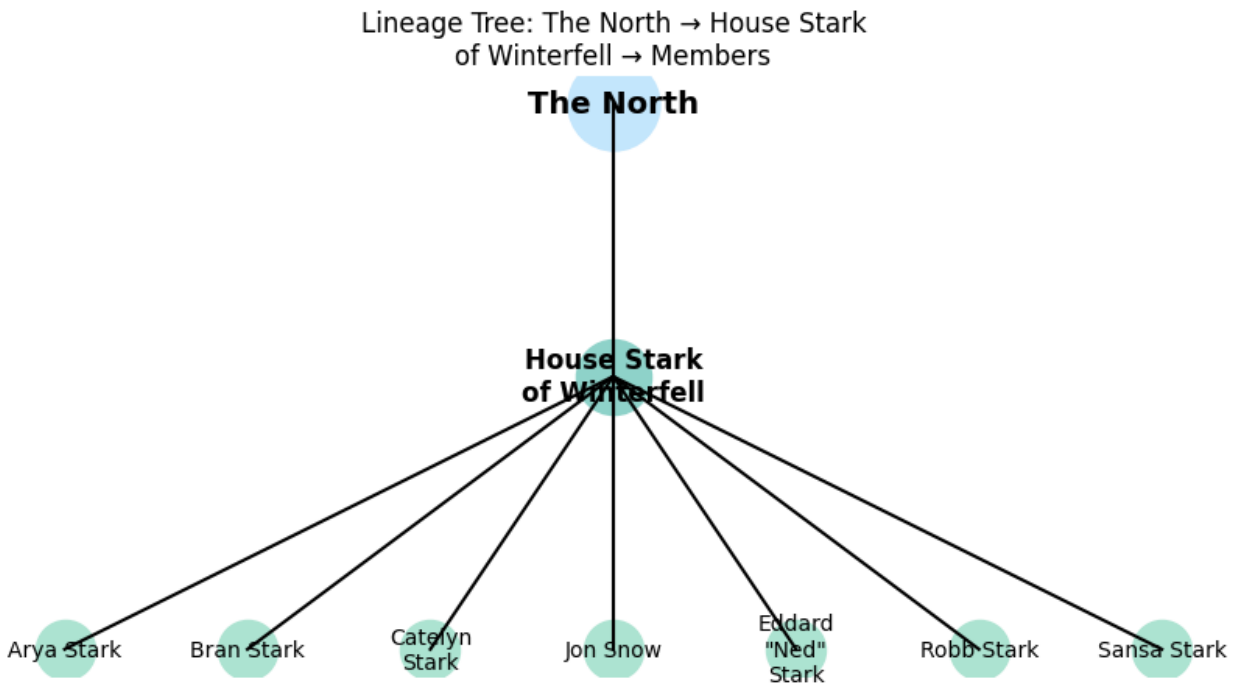


4.6.6 Tree Structure Step 3: Region, House and Members

Expanding further, we pick a specific region or house and plot all its member characters. This tree is generated for clarity by focusing only on "House Stark."

```
# For a single house and its members; labels auto-wrapped
region = "The North"
house_name = "House Stark of Winterfell" # Replace as desired
members = [c['name'] for c in db.characters.find({'house_name': house_name})]

plt.figure(figsize=(10,5))
plt.scatter(0, 0, s=1800, color='#C3E6FC')
plt.text(0, 0, wrap_label(region, 13), ha='center', va='center', fontsize=14, weight='bold')
plt.scatter(0, -1, s=1200, color='#8ED2C9')
plt.text(0, -1, wrap_label(house_name, 14), ha='center', va='center', fontsize=12, weight='bol
plt.plot([0, 0], [0, -1], 'k-')
x_start = -0.3*(len(members)-1)
for i, m in enumerate(members):
    px = x_start + 0.6*i
    plt.scatter(px, -2, s=750, color='#ACE3D1')
    plt.text(px, -2, wrap_label(m, 11), ha='center', va='center', fontsize=10)
    plt.plot([0, px], [-1, -2], 'k-')
plt.axis('off')
plt.title(f"Lineage Tree: {wrap_label(region, 13)} → {wrap_label(house_name, 14)} → Members")
plt.show()
```


Lineage Tree: The North → House Stark of Winterfell → Members
The North
House Stark of Winterfell
Arya Stark
Bran Stark
Catelyn Stark
Jon Snow
Eddard "Ned" Stark
Robb Stark
Sansa Stark

4.7 Datastructure

4.7.1 UML class diagram

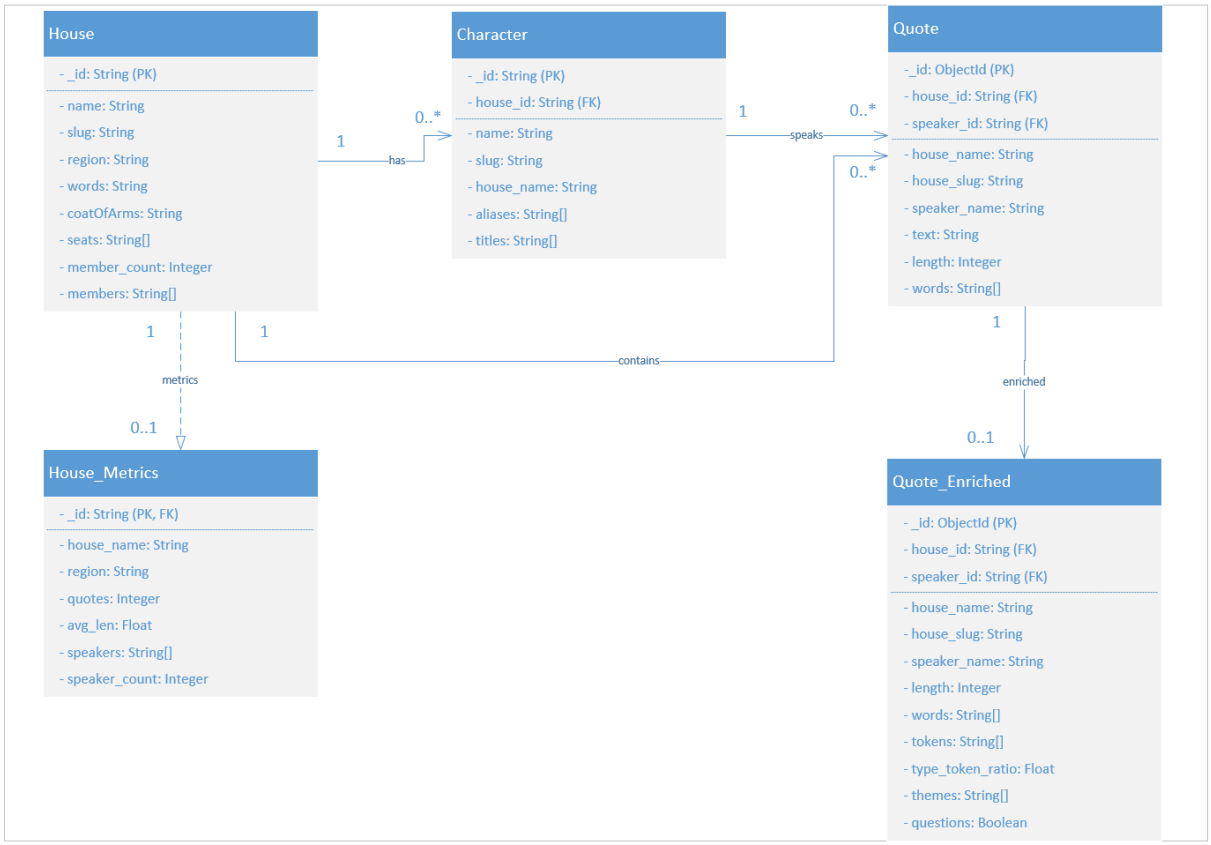


Figure 2: Class diagram

This class diagram visually describes the structure of the MongoDB database for Game of Thrones quotes analytics. The model includes: - House: Represents a noble house, with attributes like name, region, words (motto), and references to its members. - Character: Each character is uniquely identified and linked to a House via house_id. Characters can have aliases and titles. - Quote: Each quote record contains the text, length, speaker information, and is linked to both the speaker (a Character) and their House. - Quote_Enriched: An enhanced version of a quote, containing additional NLP-derived attributes such as tokens, type-token ratio, themes, and a question flag. - House_Metrics: Aggregated metrics for each house, such as number of quotes, average quote length, and list of speakers, referenced by the house's id.

Relationships: - One House has many Characters (1-to-0..*)* - *One Character speaks many Quotes (1-to-0..*), and each Quote is spoken by exactly one Character (shown by speaker_id) - One House

contains many Quotes (indirectly, via Character and Quote) - Each Quote may have one Enriched Quote (1-to-0..1) - Each House may have one House_Metrics record (1-to-0..1)

Foreign key fields are shown as associations, and multiplicities reflect the true cardinalities from your data model.

4.7.2 Entity Relationship Diagram Chen Notation

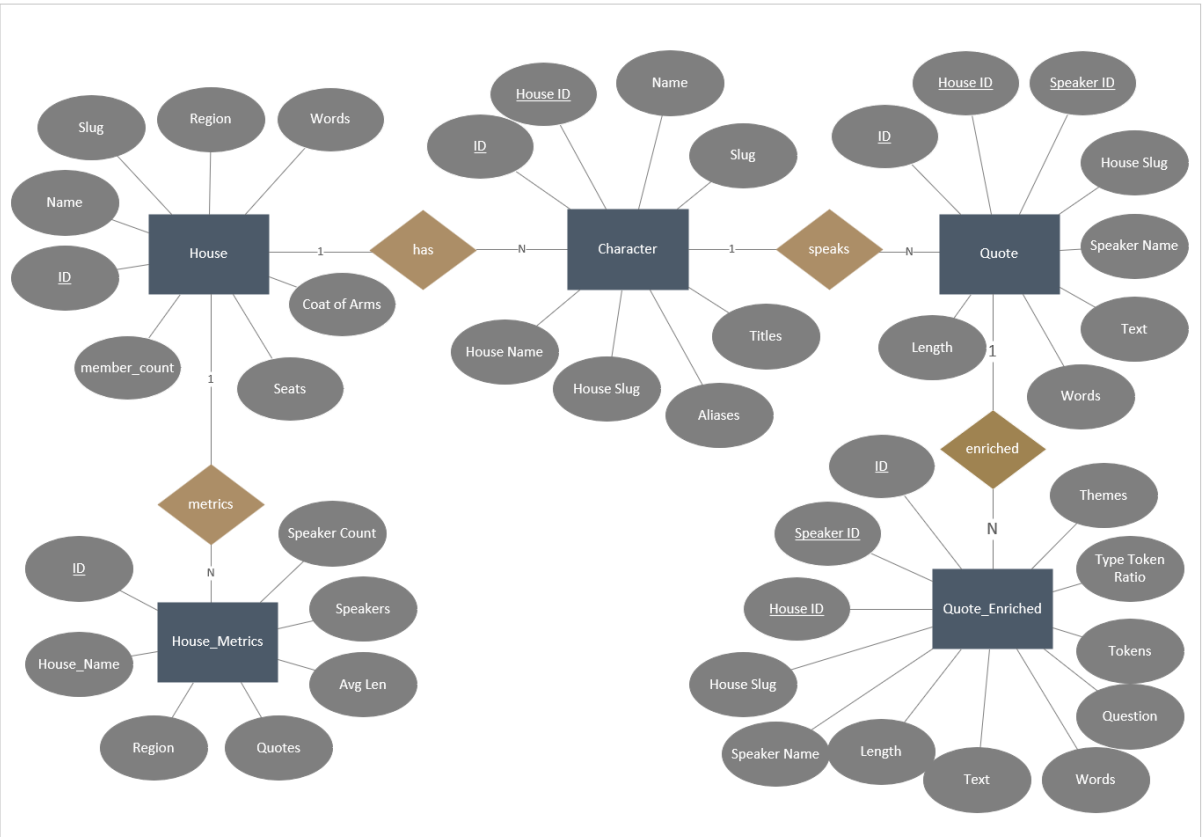


Figure 3: Entity Relationship Model Chen Notation

4.7.3 ERD 3NF

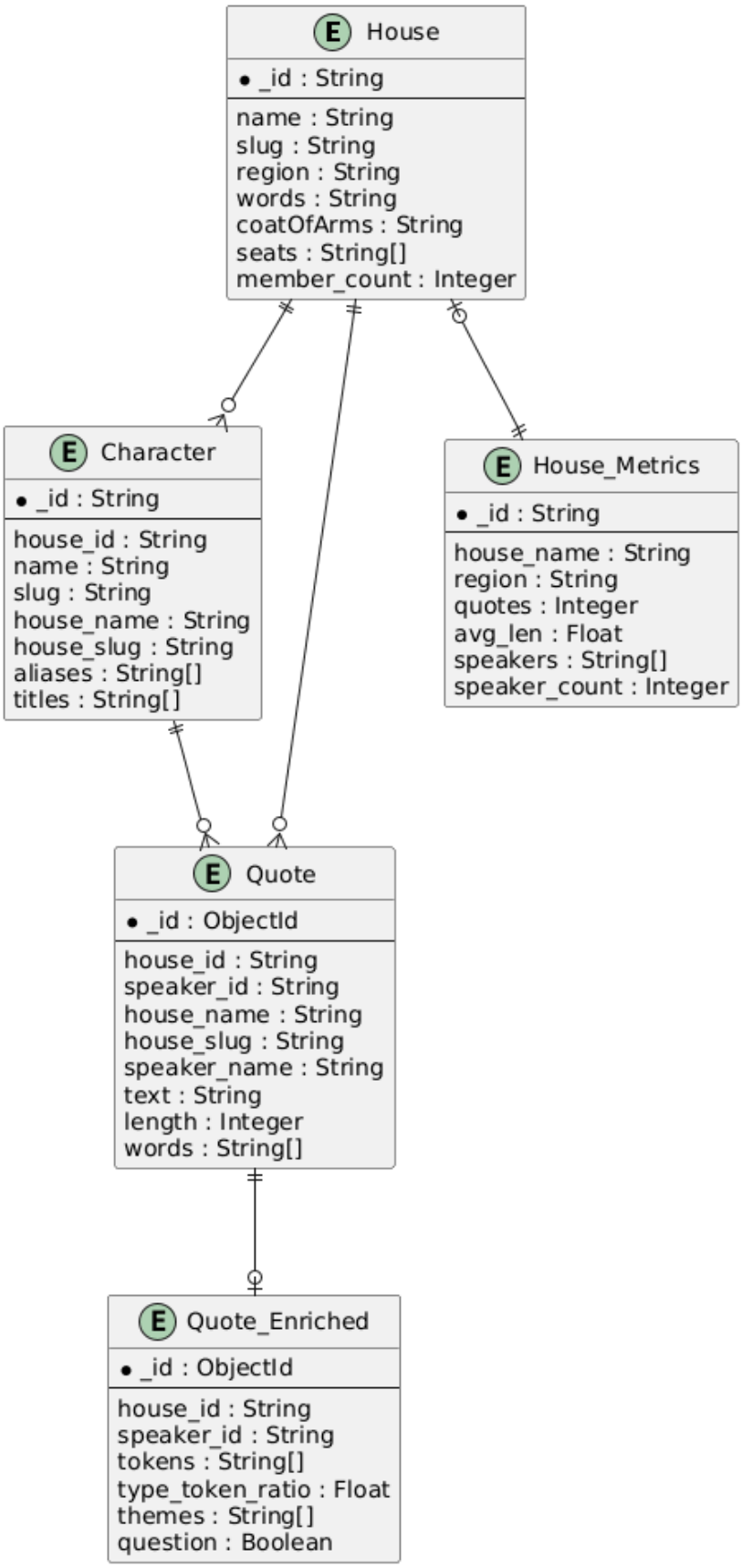


Figure 4: Entity Relationship Model

5 Data analysis

The analysis section focuses on character-driven insights. It identifies the most frequently quoted characters, examines vocabulary richness, and measures the length and structure of quotes across houses. Thematic analysis reveals recurring motifs such as power, winter, and honor, while linguistic features like co-occurrence and type-token ratios highlight differences in speaking style. Together, these analyses showcase how data engineering and NLP techniques can be applied to narrative text.

5.1 Top quoted characters

```
res = list(db.quotes.aggregate([
  {"$group":
   {"_id": "$speaker_name", "quotes":{"$sum":1}}},
  {"$sort": {"quotes":-1}}, {"$limit":15}
]))
pd.DataFrame(res)
```

	_id	quotes
0	Tyrion Lannister	26
1	Cersei Lannister	11
2	Sandor Clegane	9
3	Jon Snow	7
4	Eddard ”Ned” Stark	7
5	Tywin Lannister	7
6	Daenerys Targaryen	6
7	Petyr Baelish	5
8	Jaime Lannister	5
9	Sansa Stark	5
10	Arya Stark	4
11	Samwell Tarly	3
12	Brienne of Tarth	3
13	Theon Greyjoy	3
14	Oberyn Martell	3

5.2 Average quote length by house

```
res = list(quotes.aggregate([
  {"$match": {"house_name": {"$ne": None}}},
  {"$group": {"_id": "$house_name",
              "avg_len": {"$avg": "$length"},
              "n": {"$sum": 1}}},
  {"$sort": {"avg_len": -1}}
]))
pd.DataFrame(res)
```

	_id	avg_len	n
0	House Martell of Sunspear	114.000000	3
1	House Tarth of Evenfall Hall	101.000000	3
2	House Clegane of Clegane's Keep	98.666667	9
3	House Tyrell of Highgarden	89.333333	3
4	House Lannister of Casterly Rock	75.591837	49
5	House Baratheon of Dragonstone	74.333333	3
6	House Bolton of the Dreadfort	72.000000	1
7	House Baelish of Harrenhal	70.800000	5
8	House Tarly of Horn Hill	67.000000	3
9	House Mormont of Bear Island	66.500000	2
10	House Stark of Winterfell	66.464286	28
11	House Greyjoy of Pyke	63.666667	3
12	House Targaryen of King's Landing	59.000000	8
13	House Frey of the Crossing	48.000000	1

5.3 Characters with the richest vocabulary (type/token ratio)

```
res = list(quotes.aggregate([
  {"$match": {"words": {"$exists": True}}},
  {"$group": {"_id": "$speaker_name", "bags": {"$push": "$words"}}},
  {"$project": {
     "uniq": {"$size": {"$setUnion": "$bags"}},
     "total": {"$sum": {"$map": {"input": "$bags", "as": "b", "in": {"$size":"$$b"}}}}
  }},
  {"$addFields": {"ttr": {"$cond": [{"$gt": ["$total", 0]}, {"$divide": ["$uniq", "$total"]}, 0
```

```
  {"$sort": {"ttr": -1}}, {"$limit": 10}
]))
pd.DataFrame(res)
```

	_id	uniq	total	ttr
0	Hodor	1	3	0.333333
1	Aerys II Targaryen	1	3	0.333333
2	Joffrey Baratheon	2	15	0.133333
3	Walder Frey	1	8	0.125000
4	Tormund	1	8	0.125000
5	Ygritte	2	16	0.125000
6	Melisandre	1	8	0.125000
7	Jaime Lannister	5	47	0.106383
8	Robb Stark	1	10	0.100000
9	Joer Mormont	1	11	0.090909

5.4 Theme share by character

```
res = list(db.quotes.aggregate([
  {"$set":
   {"theme":
    {"$regexMatch": {"input": "$text",
                     "regex":"(king|queen|dragon|winter)",
                     "options":"i"}}}},
    {"$group": {"_id":"$speaker_name",
                "q_total":{"$sum":1},
                "q_theme":{"$sum": {"$cond":["$theme",1,0]}}}},
    {"$set":{"theme_share":
             {"$divide":["$q_theme","$q_total"]}}},
  {"$sort":{"theme_share":-1}}, {"$limit":15}
]))
pd.DataFrame(res)
```

	_id	q_total	q_theme	theme_share
0	Lyanna Mormont	1	1	1.000000
1	Robert Baratheon	1	1	1.000000

	_id	q_total	q_theme	theme_share
2	Robb Stark	1	1	1.000000
3	Bran Stark	2	1	0.500000
4	Sansa Stark	5	2	0.400000
5	Olenna Tyrell	3	1	0.333333
6	Brienne of Tarth	3	1	0.333333
7	Daenerys Targaryen	6	2	0.333333
8	Eddard "Ned" Stark	7	2	0.285714
9	Cersei Lannister	11	3	0.272727
10	Arya Stark	4	1	0.250000
11	Sandor Clegane	9	2	0.222222
12	Tywin Lannister	7	1	0.142857
13	Jon Snow	7	1	0.142857
14	Joffrey Baratheon	2	0	0.000000

5.5 Mentions of themes

```
res = list(quotes.aggregate([
  {"$match":
   {"text":
    {"$regex": "(king|queen|dragon|winter)", "$options":"i"}}},
  {"$group":
   {"_id":
    "$house_name", "mentions": {"$sum": 1}}}, {"$sort": {"mentions": -1}}
]))
pd.DataFrame(res)
```

	_id	mentions
0	House Stark of Winterfell	8
1	House Lannister of Casterly Rock	4
2	House Clegane of Clegane's Keep	2
3	House Targaryen of King's Landing	2
4	House Mormont of Bear Island	1
5	House Tyrell of Highgarden	1
6	House Baratheon of Dragonstone	1
7	House Tarth of Evenfall Hall	1

5.6 Longest quotes

```
res = list(db.quotes.aggregate([
  {"$addFields": {"len": {"$strLenCP": "$text"}}},
  {"$sort": {"len": -1}},
  {"$group": {
      "_id": "$speaker_name",
      "longest_len": {"$first": "$len"},
      "longest_text": {"$first": "$text"}
  }},
  {"$sort": {"longest_len": -1}},
  {"$limit": 10}
]))
pd.DataFrame(res)
```

	_id	longest_len	longest_text
0	Sandor Clegane	220	Thoros isn't here anymore, so I hope you're no...
1	Davos Seaworth	196	I may not know the north, but I know men. They...
2	Oberyn Martell	156	'Tis a big and beautiful world. Most of us liv...
3	Tyrion Lannister	151	I wish I was the monster you think I am! I wis...
4	Tywin Lannister	150	He's a Lannister! Maybe the lowest of the Lann...
5	Olenna Tyrell	146	I was told you were drunk, impertinent and tho...
6	Jaqen H'ghar	136	If the day comes when you must find me again, ...
7	Robert Baratheon	134	I'm the King, I get what I want. I never loved...
8	Lord Varys	130	Power resides where men believe it resides. It...
9	Bran Stark	129	We don't have time for all this. The Night Kin...

5.7 Average words per quote by character

```
res = list(db.quotes.aggregate([
  {"$match": {"words": {"$exists": True}}},
  {"$addFields": {"word_count": {"$size": "$words"}}},
  {"$group": {"_id": "$speaker_name",
              "avg_words": {"$avg": "$word_count"},
              "quotes": {"$sum": 1}}},
  {"$match": {"quotes": {"$gte": 3}}},  # avoid 1-quote outliers
  {"$sort": {"avg_words": -1}},
```

```
  {"$limit": 15}
]))
pd.DataFrame(res)
```

	_id	avg_words	quotes
0	Oberyn Martell	24.333333	3
1	Brienne of Tarth	21.666667	3
2	Sandor Clegane	19.333333	9
3	Tyrion Lannister	16.461538	26
4	Tywin Lannister	16.428571	7
5	Olenna Tyrell	15.000000	3
6	Jon Snow	14.428571	7
7	Petyr Baelish	14.400000	5
8	Cersei Lannister	13.363636	11
9	Samwell Tarly	12.666667	3
10	Sansa Stark	12.400000	5
11	Daenerys Targaryen	12.000000	6
12	Theon Greyjoy	12.000000	3
13	Eddard "Ned" Stark	11.428571	7
14	Arya Stark	11.250000	4

5.8 Co-occurrence of words inside quotes

```
res = list(db.quotes.aggregate([
  {"$match": {"words": {"$exists": True}}},
  {"$project": {"pairs": {
      # make small windows of word pairs (skip identical/doubles)
      "$reduce": {
        "input": {"$range": [0, {"$size": "$words"}]},
        "initialValue": [],
        "in": {"$concatArrays": ["$$value", {
          "$map": {
            "input": {"$slice": ["$words", {"$add": ["$$this", 1]}, 4]},
            "as": "w2",
            "in": {"$concat": [
              {"$arrayElemAt": ["$words", "$$this"]}, "|", "$$w2"
            ]}
```

```
          }
        }]}
      }
  }}},
  {"$unwind": "$pairs"},
  {"$group": {"_id": "$pairs", "w": {"$sum": 1}}},
  {"$sort": {"w": -1}},
  {"$limit": 40}
]))
pd.DataFrame(res[:10])
```

	_id	w
0	the|the	19
1	the|of	13
2	the|and	12
3	and|the	12
4	i|i	10
5	the|you	9
6	of|the	9
7	i|to	9
8	a|the	8
9	a|a	8

5.9 Full-text search: top speakers for winter

```
res = list(db.quotes.aggregate([
    {"$match": {"$text": {"$search": "winter"}}},
    {"$group": {"_id": "$speaker_name", "hits": {"$sum": 1}}},
    {"$sort": {"hits": -1}},
    {"$limit": 10}
]))
pd.DataFrame(res)
```

	_id	hits
0	Eddard "Ned" Stark	1
1	Arya Stark	1

6 Conclusions

The project demonstrates the effectiveness of a MongoDB-based ELT pipeline for text-centric datasets. By combining structured and unstructured sources, it enables flexible querying and scalable transformations. The results highlight not only the most dominant characters in dialogue but also linguistic nuances that reflect personality and context within the series.

7 Learnings

Throughout the project, several technical and analytical insights were gained: - Efficient use of MongoDB for semi-structured data management. - Integration of external APIs for data enrichment. - Application of Python-based transformations for text analytics. - Importance of clean data models for reproducible analyses. - Practical experience in designing an end-to-end ELT pipeline.

```
%%HTML
<style>
/* display:none  -> hide In/Out column */
/* display:block -> show In/Out column */
div.prompt {display:none}
h1#Table-of-Contents {display:none}
</style>
```

```
<IPython.core.display.HTML object>
```